Parallel balanced minimum evolution search in Phylo2Vec coordinates

Abstract

Balanced minimum evolution (BME) provides a distance-based criterion for phylogenetic inference, but searching large tree spaces requires repeated evaluation of many candidate rearrangements. We develop a shared evaluator that computes all valid single-coordinate BME changes for an N-leaf tree in an ordered Phylo2Vec representation in $O(N^2)$ work and supports parallel GPU execution. For larger trees, we optimize the connections between fixed components on a smaller skeleton tree. Balanced averages of the original distances make the skeleton objective equal to the full-tree objective up to a constant. Every skeleton change therefore gives the corresponding full-tree BME change, with an $O(K^2)$ coordinate sweep once the distances between its K components are available. On simulated RNASim ($L = 1{,}596$) and Jukes–Cantor ($L = 1{,}000$ and $10{,}000$) alignments from $N = 1{,}000$ to $10{,}000{,}000$ leaves, GPU full-tree and minibatch refinement match or improve the topological accuracy and BME scores of 64-thread minimum-evolution baselines while reducing refinement time by one to two orders of magnitude. At $N = 10{,}000{,}000$, single-GPU minibatch refinement on one NVIDIA H200 completes in 543.7 s (1.73 GB host RAM, 10.08 GB VRAM) on RNASim and 2,483.4 s (2.16 GB host RAM, 60.37 GB VRAM) on 10,000-site alignments. The reduction preserves the specified BME objective; numerical evaluation uses floating-point arithmetic, and topology search remains heuristic.

1 Introduction

Distance-based phylogenetic inference first estimates pairwise evolutionary distances and then searches for a tree that explains them. Balanced minimum evolution (BME) scores a topology using a weighted sum of those distances, with weights determined by the branching structure along each leaf-to-leaf path [1, 2, 3]. The computational challenge is to evaluate many possible rearrangements repeatedly as the tree changes. Classical subtree-prune-and-regraft (SPR) recurrences already permit quadratic-time neighborhood sweeps [4, 5]. At large scales, their practical cost depends on how calculations are shared, how distance tables are stored, and how work is distributed across processors.

For a tree with N leaves, we organize BME search around a table of coordinate changes in the Phylo2Vec representation. Each entry gives the change in the objective when one integer in the tree's encoding is replaced. The table is evaluated through shared subtree-distance calculations, exposing independent work for parallel execution. Relabeling the current tree changes which rearrangements are represented by single-coordinate updates while preserving the tree and its objective.

The same evaluator operates on a smaller tree when some internal structure is held fixed. We replace each fixed component by one skeleton leaf and retain its contribution through an exact balanced average of the original distances. The full objective then equals the skeleton objective plus a constant that depends only on the fixed components. Consequently, every change evaluated on the skeleton has the same BME change in the full tree. We call this block of component connections a *minibatch*: it restricts the structures being optimized while retaining all leaves in the objective calculation.

We develop three algorithmic contributions:

1. **Shared coordinate evaluation.** All $(N-1)^2$ valid entries are computed in $O(N^2)$ work, matching the size of the explicit table. A map from coordinate values to regraft edges avoids separately scoring candidate trees.

2. **BME preservation in blocks.** The same computation applies to a K-component skeleton in $O(K^2)$ work after its balanced distances are available. The reduction includes the entire complement of a subtree window and specifies the updates required between windows.

3. **Parallel execution.** Independent distance columns and SPR rows support GPU distribution and selection of the best update in each row. Reconstructing rows when needed reduces the balanced-distance table from approximately $4N^2$ to N^2 entries while retaining quadratic sweep work.

The contribution builds on established representation and BME results. Phylo2Vec introduced coordinate differences and ordered relabeling [6, Section 1.3.2]. Penn et al. [7, Lemmas 0.9–0.11] prove that every rooted binary tree has an ordered representation after relabeling. Each coordinate replacement either leaves the tree unchanged or applies a single SPR move (Lemma A.2). Our evaluator retains the quadratic sweep complexity of classical BME SPR calculations. Balanced subtree averages [2, 3] and the fixed-clade reduction of the BME polytope [10, Section 2.4] provide the basis for exact skeleton evaluation. We state the objective identity and prove the conditions needed by the block algorithm.

GradME uses a cubic-time recurrence and automatic differentiation to optimize an expected BME objective over ordered coordinates [7]. Here evaluation is specialized to a discrete tree, including its full coordinate range.[1] Parallel taxon-addition BME [8], GPU distance-based placement and divide-and-conquer tree construction (DIPPER) [14], and learned SPR selection [9] address tree construction and search policy, respectively. Our experiments evaluate end-to-end tree refinement across sequence alignments from $N = 1{,}000$ to 10,000,000 leaves, with numerical verification and standalone sweep microbenchmarks reported in Appendix C.

2 Methods

2.1 Overview

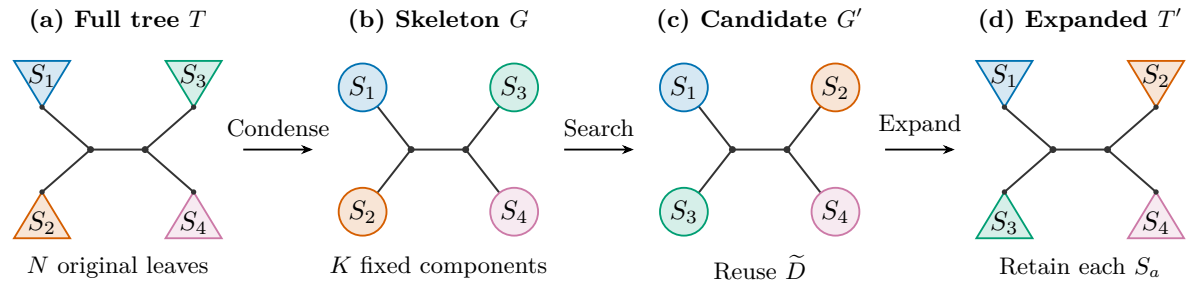
The method takes a tree and a fixed matrix of distances between its leaves. A full-tree sweep computes the objective change for every valid coordinate replacement of the current encoding. A minibatch sweep applies the same evaluator to a skeleton of fixed components (Figure 1). After selecting and accepting a skeleton update, the components are expanded back into the full tree before the next block is constructed. The full-tree case is recovered by taking every component to be a single leaf. Algorithms 1 and 2 specify the complete optimisation procedure in Section 2.6.

The minibatch reduction preserves full-tree BME score differences in real arithmetic. The distances may be estimates, numerical evaluation uses floating-point arithmetic, and topology search is heuristic.

2.2 BME and Phylo2Vec coordinates

Let T be an unrooted binary tree on $N \geq 3$ labeled leaves, and let D be a fixed, finite, symmetric matrix with zero diagonal. Write $q_T(x, y)$ for the number of internal degree-three vertices on the

[1] At a point mass on an ordered v, transferring probability t from v_i to $j \leq i$ mixes the scores as $(1-t)f(v) + tf(v^{(i \leftarrow j)})$, where $f(v) = L(T(v); D_\pi)$, giving derivative $\Delta_{i,j}$. The quadratic bound applies at discrete trees; gradients at interior distributions are outside its scope.



Balanced component distances $\widetilde{D}_{ab} = \sum_{x \in S_a} \sum_{y \in S_b} \omega_a(x)\omega_b(y) D_{xy} \quad (a \neq b)$

Identical objective changes $L(T'; D) - L(T; D) = L(G'; \widetilde{D}) - L(G; \widetilde{D})$

Figure 1: Block evaluation preserving BME, illustrated with $K = 4$ fixed components. **(a)** Triangles represent components S_a of the full tree, each attached at its apex. **(b)** Each component becomes one skeleton leaf. The condensed distance $\widetilde{D}_{ab}$ retains all leaf pairs between S_a and S_b, with balanced weights ω obtained by halving mass at each binary split. **(c)** Search changes the connections between components while reusing $\widetilde{D}$. **(d)** Expanding the candidate preserves each component's internal topology and attachment root. Within-component contributions cancel, so the skeleton and full-tree objective changes agree. A proper subtree window includes its entire complement as an additional component.

path from x to y. The BME objective is

$$L(T; D) = \sum_{0 \leq x < y < N} D_{xy}\, 2^{-q_T(x,y)}. \tag{1}$$

If the path has $e_T(x, y)$ edges, then $q_T(x, y) = e_T(x, y) - 1$. Thus the unordered-pair coefficient is $2^{1-e_T(x,y)}$. An artificial degree-two root used to encode the tree is suppressed before evaluating these path lengths. No additivity assumption on D is needed for the identities below.

The Phylo2Vec domain for rooted binary trees is

$$\mathcal{V}_N = \{(v_0, \ldots, v_{N-2}) : 0 \leq v_i \leq 2i\}, \qquad |\mathcal{V}_N| = (2N - 3)!!. \tag{2}$$

We use the ordered, or cherry-first, subset $\mathcal{V}_N^{\mathrm{cf}} = \{v : 0 \leq v_i \leq i\}$ as the baseline for coordinate evaluation. Every rooted binary tree can be represented in this subset after a suitable leaf permutation [7, Lemmas 0.9–0.11]. Appendix A.2 gives the child-order rule used here. If $\pi(a)$ is the original leaf represented by current index a, the distance matrix in these coordinates is $D_\pi[a, b] = D[\pi(a), \pi(b)]$. Permuting both axes in this way preserves the objective. Constructing the permutation takes $O(N)$ time from a decoded tree, while explicitly copying a dense permuted matrix takes $O(N^2)$ time.

Write $T(v)$ for the unrooted tree obtained by decoding v and suppressing its artificial root. For an ordered baseline v, define the discrete coordinate difference

$$\Delta_{i,j} = L(T(v^{(i \leftarrow j)}); D_\pi) - L(T(v); D_\pi), \qquad 0 \leq j \leq 2i. \tag{3}$$

Here $v^{(i \leftarrow j)}$ replaces only entry i by j. Row i of the table therefore enumerates the allowed replacements of one coordinate, and $\Delta_{i,j} < 0$ identifies an improving move. We use *discrete gradient* as a synonym for this table of finite objective differences, and *coordinate sweep* for one evaluation of the table at a fixed tree. There are $(N - 1)^2$ valid coordinate entries, including unchanged coordinates and entries that can represent the same unrooted topology. This count is not a count of distinct SPR neighbors. General candidate values $j > i$ are allowed even though

the baseline is ordered. After accepting such a move, the resulting tree is decoded and relabeled before the next ordered evaluation. Relabeling changes computational indices and permutes both axes of D together, so the observations remain attached to their original leaves.

2.3 Quadratic-time coordinate evaluation

The evaluator first constructs a shared table of subtree distances, then propagates score changes across regraft positions. For distinct vertices u, w, let $A(u, w)$ be the balanced mean distance under D_π between the components rooted at u, w and directed away from their connecting path. If a, b are the two neighbours of an internal vertex u away from w, equal weighting of the two branches gives

$$A(u, w) = \tfrac{1}{2}\{A(a, w) + A(b, w)\}. \tag{4}$$

For a leaf x, initialize $A(y, x) = D_\pi[y, x]$ at the other leaves and apply Equation (4) towards x, processing children before their parent. Each of the $N - 2$ internal entries is one average of two previously computed values, so the entire column costs $O(N)$. Once all leaf columns are complete, symmetry supplies the leaf entries $A(x, w) = A(w, x)$ for each internal target w. The same recurrence computes the entries in its three incident branches in $O(N)$ operations; the diagonal is unused. Thus all $2N - 2$ columns cost $O(N^2)$, including on unbalanced trees. Appendix A.5 describes the implementation with a fixed computational root.

For each coordinate $i \geq 1$, varying v_i regrafts a fixed subtree P_i (Lemma A.2). Moving its attachment between adjacent edges is a nearest-neighbour interchange (NNI). For the four components P, Q, R, S surrounding the current NNI edge, let d_{UV} denote the balanced mean distance between U, V at their attachment roots. The interchange $PQ \mid RS \to PR \mid QS$ changes the BME score by

$$\delta_{\mathrm{NNI}} = \tfrac{1}{4}(d_{PR} + d_{QS} - d_{PQ} - d_{RS}). \tag{5}$$

The standard SPR recurrence accumulates these increments along the remaining tree, obtaining the changing component means from A and a constant number of running averages and weights per traversal state [4]. Each edge therefore costs $O(1)$ operations, and a single traversal scores all regraft positions for P_i in $O(N)$. The edge-to-coordinate map in Appendix A.4 extracts the scores for the $2i + 1$ coordinate values.

Proposition 2.1 (Quadratic coordinate evaluation)**.** *Given an ordered N-leaf encoding v, its leaf permutation π, and constant-time access to D_π, all $(N-1)^2$ entries in Equation (3) can be evaluated using $O(N^2)$ arithmetic operations and $O(N^2)$ storage. In real arithmetic, every entry equals the corresponding full-tree BME difference.*

Proof. Constructing A costs $O(N^2)$. For each row $i = 1, \ldots, N - 2$, the SPR traversal and coordinate mapping take $O(N)$ operations, while $\Delta_{0,0} = 0$. The total is therefore

$$\underbrace{O(N^2)}_{\text{construct } A} + \underbrace{(N-2)\,O(N)}_{\text{evaluate coordinate rows}} = O(N^2).$$

The stored tables have $O(N^2)$ entries. The SPR recurrence gives changes relative to the original attachment, and the edge map in Appendix A.4 identifies them with the corresponding coordinate replacements. □

Every candidate topology is either unchanged or an SPR neighbour; the coordinate table need not cover the full SPR neighbourhood.

2.4 Condensed distances and BME preservation

Partition all N leaves into $K \geq 3$ fixed components $S_1, \ldots, S_K$. Each component has a fixed internal binary topology and a single attachment to the rest of the tree. A component can consist

of one leaf. For a non-singleton component, let r_a be the internal vertex adjacent to its attachment edge, and let $h_a(x)$ count the branching vertices from r_a to leaf x, including r_a. For a singleton, set $h_a(x) = 0$. Define its balanced leaf weights by

$$\omega_a(x) = 2^{-h_a(x)}, \qquad x \in S_a. \tag{6}$$

The weights sum to one within each component (Lemma A.1 in Appendix A.1).

Let G be an unrooted binary skeleton with one leaf for each component. Construct T_G by replacing skeleton leaf a with S_a, retaining the attachment edge. Define a condensed distance matrix by

$$\widetilde{D}_{ab} = \sum_{x \in S_a} \sum_{y \in S_b} \omega_a(x)\omega_b(y)D_{xy} \quad (a \neq b), \qquad \widetilde{D}_{aa} = 0. \tag{7}$$

These are exact balanced averages of the original pairwise distances. They can be formed by the weighted sum above or, equivalently, by recursively averaging over the two children of each component root. All leaf contributions are retained. This is an objective-level form of the fixed-clade reduction discussed by Haws et al. [10, Section 2.4]. We include the proof to specify the weights and the scope of exactness needed by the algorithm.

Theorem 2.2 (BME decomposition over fixed components)**.** *There is a constant C, independent of the skeleton topology, such that*

$$L(T_G; D) = C + L(G; \widetilde{D}). \tag{8}$$

Consequently, for any two skeletons G and G',

$$L(T_{G'}; D) - L(T_G; D) = L(G'; \widetilde{D}) - L(G; \widetilde{D}). \tag{9}$$

The proof in Appendix A.1 separates the fixed within-component terms from the between-component terms, whose weights factor through the skeleton.

The constant C contains the contributions of pairs within the same fixed component. It cancels when comparing skeletons, so the reduction preserves every objective change even though the skeleton's absolute score omits C. The result applies to every reconnection that preserves the components and to arbitrary fixed dissimilarities, including non-additive estimated distances.

Corollary 2.3 (Exact minibatch coordinate differences)**.** *Let v encode an ordered skeleton G, keep the attached components fixed, and index $\widetilde{D}$ by the same leaf labeling. Evaluating a coordinate replacement $v_i \leftarrow j$ using the exact matrix $\widetilde{D}$ gives*

$$\Delta^{\mathrm{skel}}_{i,j} = L(T_{G(v^{(i \leftarrow j)})}; D) - L(T_{G(v)}; D). \tag{10}$$

Once $\widetilde{D}$ is available, all valid skeleton coordinate differences can be evaluated in $O(K^2)$ time.

Proof. Apply Proposition 2.1 to the K-leaf skeleton with input $\widetilde{D}$. Equation (9) identifies each resulting skeleton difference with the corresponding full-tree difference. □

Corollary 2.4 (A proper subtree window)**.** *For a window attached to the rest of a binary tree by one edge, treat its entire complement as an additional component oriented away from that edge. If its internal topology and exact balanced distances to the other components are retained, Equation (9) applies to rearrangements of the window skeleton.*

Proof. Cutting the attachment edge produces two components. The endpoint in the complement has two outward branches unless the complement is a singleton. It therefore defines precisely a binary component of the type used in the theorem. Including it completes the partition of all leaves, so both within-window and window-to-complement contributions are included. □

The outside component retains distances between leaves inside and outside the window. Deleting the complement would change the objective being optimized. The weights entering the outside distances are determined by the current topology of the complement and its attachment edge. They must be updated when a preceding rearrangement changes that topology.

2.5 Minibatch evaluation and sequential updates

With the fixed components and exact condensed matrix in place, candidate skeletons are evaluated against the full-tree objective through Corollary 2.3. The same matrix can be reused throughout a skeleton search because its component topologies and attachment roots remain fixed. For a candidate G', let $\mathcal{P}(G, G') = \{(a, b) : a < b,\ q_G(a, b) \neq q_{G'}(a, b)\}$. Its full-tree BME change can also be computed directly from the changed pairs:

$$\delta = \sum_{(a,b)\in\mathcal{P}(G,G')} \left(2^{-q_{G'}(a,b)} - 2^{-q_G(a,b)}\right) \tilde{D}_{ab}. \tag{11}$$

Unchanged pairs contribute zero, and the within-component constant cancels. Theorem 2.2 therefore identifies δ with the full-tree BME change. Thus both coordinate selection and the evaluation of a combined candidate can use the same exact condensed distances.

Windows are committed sequentially. After a commit, affected ancestor summaries are updated and the next window's outside distances are constructed from the current tree. Node-disjoint windows need not have additive BME changes because both updates can alter the coefficients of distances between their leaves. For consecutive accepted trees $T^{(0)}, \ldots, T^{(M)}$, exact arithmetic gives

$$L(T^{(M)}; D) - L(T^{(0)}; D) = \sum_{m=1}^{M} \left[L(T^{(m)}; D) - L(T^{(m-1)}; D)\right]. \tag{12}$$

If each increment is negative, so is the cumulative increment. This argument requires that each acceptance calculation uses the current tree, rather than a shared pre-update tree.

On sufficiently deep trees, finite-precision leaf weights can also underflow.

2.6 Optimisation algorithm

Algorithm 1 separates block construction from candidate search. Its inputs are a starting tree T_0, fixed distances D, a schedule $\mathcal{W}$, and a skeleton solver $\mathcal{A}$. Skeleton returns a partition $\mathcal{S}$ of all leaves into fixed rooted components and their current skeleton G. For a proper subtree window, $\mathcal{S}$ includes the entire oriented complement. Weights constructs their balanced weights $\Omega = \{(S_a, \omega_a)\}_{a=1}^{K}$. Condense evaluates Equation (7) without truncation, and Expand restores the same components with their original leaf identities. The components and their attachment roots remain fixed during each solver call. Full-tree search is the special case of one block containing N singleton components.

Algorithm 1 Full-tree and minibatch BME search

Input: T_0, D, schedule $\mathcal{W}$, solver $\mathcal{A}$, block and sweep budgets B, I, margin $\varepsilon \geq 0$.
Output: tree T and accumulated score change s.

		Execution
1	$T \leftarrow T_0$; $s \leftarrow 0$	CPU
2	**for** $b = 0, \ldots, B-1$ **do**	CPU
3	$w \leftarrow \mathcal{W}(T, b)$; **if** $w = \perp$ **then break**	CPU
4	$(G, \mathcal{S}) \leftarrow$ Skeleton(T, w)	CPU
5	$\Omega \leftarrow$ Weights$(\mathcal{S})$	CPU
6	$M \leftarrow$ Condense(Ω, D)	GPU
7	$G' \leftarrow \mathcal{A}(G, M; I)$	CPU+GPU
8	$\delta \leftarrow L(G'; M) - L(G; M)$	CPU+GPU
9	**if** $\delta < -\varepsilon$ **then**	CPU
10	$T \leftarrow$ Expand$(G', \mathcal{S})$; $s \leftarrow s + \delta$	CPU
11	Refresh(T)	CPU
12	**return** (T, s)	CPU

Execution labels refer to the GPU implementation. CPU+GPU denotes a mixed stage.

Each block is constructed from the current tree. Refresh updates affected ancestor summaries, and outside distances are rebuilt before the next window. The acceptance test evaluates the joint candidate, including interactions between its coordinate updates. By Theorem 2.2, every committed change decreases the full-tree objective in real arithmetic. A schedule may expose overlapping local windows and larger backbone connections. Its construction and implementation parameters are specified in Appendix B.

Algorithm 2 gives the CPU skeleton solver. The persistent state (v, π) contains the encoding and its map to original component identities, with $M_\pi[a, b] = M[\pi(a), \pi(b)]$. Normalise leaves an ordered encoding unchanged and otherwise relabels it, composing the permutation. Sweep returns the current score and coordinate differences. RankMin sorts the improving row minima. BatchStep tests compatible batches by their joint score, halves a rejected batch, and falls back to the best single move. Best selects the lower-scoring tree, retaining its first argument on a tie.

Algorithm 2 CPU skeleton search

Input: G_0, component matrix M, sweep and escape budgets I, E, margin rule $\epsilon(\ell)$.
Output: best scored skeleton in its original component labeling.

$(v, \pi) \leftarrow \mathsf{Order}(G_0)$; $G_\star \leftarrow G_0$; $r \leftarrow 0$; $e \leftarrow 0$
for $t = 1, \ldots, I$ **do**
  $(v, \pi) \leftarrow \mathsf{Normalise}(v, \pi)$
  $(\ell, \Delta) \leftarrow \mathsf{Sweep}(v, M_\pi)$; $\eta \leftarrow \epsilon(\ell)$
  $G_\star \leftarrow \mathsf{Best}(G_\star, \mathsf{Decode}(v, \pi); M)$
  $R \leftarrow \mathsf{RankMin}(\Delta, -\eta)$
  if $R \neq \varnothing$ **then**
    $v \leftarrow \mathsf{BatchStep}(v, M_\pi, R, \eta)$; $r \leftarrow 0$; **continue**
  $u \leftarrow \mathsf{ExtraSPR}(v, M_\pi, \eta)$
  if $u = \perp$ **and** $r = 0$ **then**
    $(v, \pi) \leftarrow \mathsf{Reverse}(v, \pi)$; $r \leftarrow 1$; **continue**
  if $u = \perp$ **then** $u \leftarrow \mathsf{RemainingSPR}(v, M_\pi, \eta)$
  if $u = \perp$ **and** $e < E$ **and** $t < I$ **then**
    $e \leftarrow e + 1$; $u \leftarrow \mathsf{Escape}(v, \Delta, e, \eta)$
  if $u = \perp$ **then break**
  $v \leftarrow u$; $r \leftarrow 0$
return $\mathsf{Best}(G_\star, \mathsf{Decode}(v, \pi); M)$

Here $\perp$ denotes failure to find a candidate, and r records whether the reversed ordered labeling has been tried since the last topology change. ExtraSPR checks additional regrafts for the pruned components already scanned, while RemainingSPR completes the unrooted SPR scan, including complement-side prunes. Both require a change below $-\eta$. Reverse changes the ordered labeling and composes π without changing the tree. Escape proposes a deterministic pair of uphill coordinate replacements, with the best scored tree retained across such trials. Appendix B defines the batch and escape rules.

The GPU solver is an alternative $\mathcal{A}$. Balanced-distance recurrences, row-minimum evaluation, and score reductions run on the GPU. The CPU packs the SPR proposals, updates and relabels the tree, and retains the best scored skeleton without the CPU completion scan. For the block change, the GPU supplies scores or pair-distance contributions and the CPU assembles the final difference and decides acceptance. The objective-preservation guarantee requires complete distances for acceptance and current outside information. A block or sweep budget limits the search, and only a completed unsuccessful SPR scan establishes SPR-local optimality up to the numerical margin. Neither solver guarantees a global BME optimum.

2.7 Parallel evaluation and computational cost

Equation (4) has an independent recurrence for each fixed column. Assigning columns to threads lets neighboring threads access consecutive entries while following the same traversal order. The

top-down correction for column w follows its ancestor path. Depth-first entry intervals determine the next child without a traversal stack. Its total work over columns is $O(\sum_w \text{depth}(w))$, bounded by $O(N^2)$ and reduced to $O(N \log N)$ for trees of logarithmic height.

A full balanced-distance table stores approximately $4N^2$ entries. The column-tiled variant retains the internal-to-leaf block and reconstructs required internal rows on demand. This reduces that table to approximately N^2 entries, while the input matrix and other buffers still contribute to total memory. A row-wise reduction can return the best coordinate replacement in each row without storing the full coordinate table. Columns and SPR rows can also be distributed across GPUs. Appendix A.5 gives the dependency and workspace arguments.

For a skeleton with K leaves, a coordinate sweep costs $O(K^2)$ once its distances are available. This does not make arbitrary distance construction independent of N. Direct evaluation of one condensed pair can require $|S_a||S_b|$ original distances. Explicit leaf weights use $O(N)$ entries, and constructing exact condensed distances can dominate runtime for large component pairs. When $D_{xy} = \phi(x)^\top \psi(y)$ has a fixed-dimensional bilinear representation, weighted component embeddings give $\widetilde{D}_{ab} = (\sum_x \omega_a(x)\phi(x))^\top (\sum_y \omega_b(y)\psi(y))$. That shortcut is exact for the specified bilinear distance. It does not justify averaging embeddings before applying a nonlinear sequence-distance correction.

3 Experimental design

We evaluate sequence-to-tree refinement from $N = 1{,}000$ to $N = 10{,}000{,}000$ leaves across two simulated benchmark suites with known reference topologies:

1. **RNASim ($L = 1{,}596$ sites).** Subsampled alignments from the million-taxon RNASim simulation benchmark [11] at $N = 1{,}000$, 5,000, 10,000, 50,000, 100,000, 1,000,000, and a 10,000,000-leaf extension.

2. **Jukes–Cantor ($L = 10{,}000$ and $L = 1{,}000$ sites).** Unpaginated DNA alignments simulated under the Jukes–Cantor model at $L = 10{,}000$ sites (`JC10k`) across $N = 1{,}000$ to 10,000,000 leaves, paired with $L = 1{,}000$-site alignments (`JC1k`) at $N = 10{,}000$ to isolate the effect of sequence length on distance noise and topological recovery.

For $N \leq 100{,}000$, every condition is evaluated over $R = 5$ independent replicates (rep $= 0, \ldots, 4$), reporting mean and standard deviation across replicates. At $N = 1{,}000{,}000$ and 10,000,000, single-replicate runs are reported due to baseline compute cost. All refinement methods at $N \leq 1{,}000{,}000$ start from the same initial tree produced by VeryFastTree 4.0.5 [13] with minimum-evolution and maximum-likelihood phases disabled (`VFT-InitNJ`, flags `-nome -noml`). At $N = 10{,}000{,}000$, where constructing a full neighbor-joining starting tree is prohibitive, refinement starts from a perturbed reference tree with initial normalized Robinson–Foulds (RF) error of approximately 35.9%.

We compare GPU full-tree search (`Phylo2Vec-Full`, evaluated up to $N = 10{,}000$) and GPU minibatch search (`Phylo2Vec-Minibatch`, evaluated up to $N = 10{,}000{,}000$) under gap-aware Jukes–Cantor (JC69) distances against both 64-thread CPU baselines—FastME 2.1.6 SPR refinement (`-s`) [5], FastTreeMP 2.2 [12] and VeryFastTree 4.0.5 minimum-evolution refinement (`-noml`), and reference maximum-likelihood refinement (`ME+ML`, evaluated up to $N = 10{,}000$)—and the GPU distance-based phylogenetic placer DIPPER 0.1.5 [14] (`-i m -o t -d 2 -force-binary`, evaluated up to $N = 1{,}000{,}000$, which constructs trees directly from sequences on GPU via neighbor-joining for $N < 30{,}000$, K-closest placement for $30{,}000 \leq N < 1{,}000{,}000$, and divide-and-conquer placement at $N = 1{,}000{,}000$). For trees above 2,048 leaves, the sequence minibatch implementation caps component leaf-weight lists at 64 entries during inner skeleton proposals and recomputes all affected condensed pairs over complete leaf profiles before evaluating Equation (11) in FP64 (acceptance margin 10^{-6}). At $N = 10{,}000{,}000$, packed two-bit sequences are streamed from disk to GPU memory in 256 MB chunks to bound host memory usage.

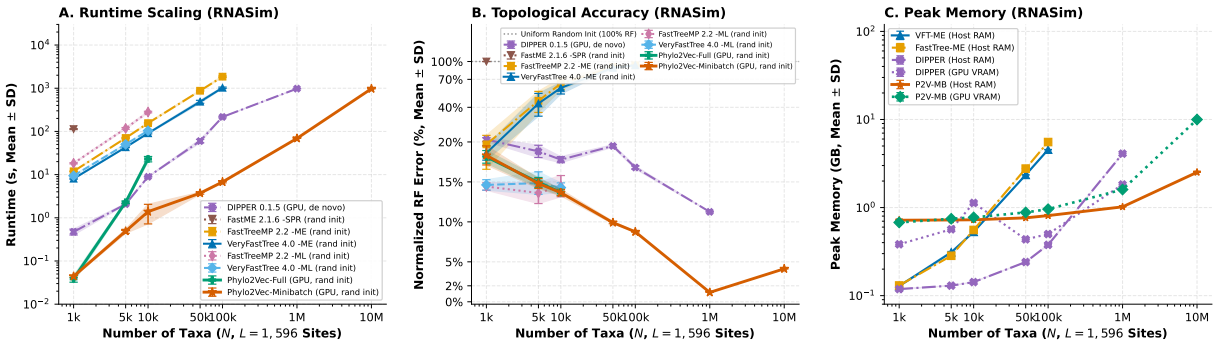


Figure 2: Sequence-to-tree refinement scaling on simulated **RNASim** alignments (L = 1,596 sites) across N = 1,000 to 10,000,000 leaves ($R = 5$ independent replicates for $N \leq 100{,}000$; points and error bars show mean $\pm$ 1 SD). **(A)** Wall-clock refinement runtime. **(B)** Normalized Robinson–Foulds topological error (%). **(C)** Peak host RAM and GPU VRAM footprint.

We record four metrics for every run: topological error measured by normalized RF distance (incorrect nontrivial splits divided by $N - 3$), wall-clock refinement time (excluding the shared `VFT-InitNJ` initialization), BME objective change ΔBME relative to the starting tree under the exact full-tree JC69 distance matrix, and peak memory footprint (maximum resident set size on the host CPU and peak allocated device memory on the GPU). Independent numerical verification of the coordinate evaluator, small-tree CPU comparisons, and standalone multi-GPU sweep microbenchmarks are reported in Appendix C.

4 Results

Figures 2 and 3 summarize runtime scaling, topological accuracy, peak memory footprint, and sequence-length sensitivity across the RNASim (L = 1,596) and Jukes–Cantor (L = 1,000 and 10,000) benchmarks from N = 1,000 to 10,000,000 leaves (complete numerical tables in Appendix C).

Three consistent trends emerge across both benchmark suites:

1. **Topological equivalence across minimum-evolution solvers.** Across all sizes where CPU minimum-evolution baselines terminate ($N \leq 100{,}000$), `Phylo2Vec-Full` and `Phylo2Vec-Minibatch` track the same normalized RF error curve as 64-thread `FastME`, `FastTreeMP` (`-noml`), and `VeryFastTree` (`-noml`) (Figures 2B and 3B), while reaching slightly lower BME objective values (Tables 1–2). By contrast, `DIPPER` constructs trees without NNI or SPR minimum-evolution refinement, leaving its topological error near or above the unrefined `VFT-InitNJ` baseline across all regime transitions. Restricting inner searches to K-component skeletons therefore preserves full-tree minimum-evolution accuracy.

2. **Runtime and memory scaling to 10^7 leaves.** Full-tree GPU coordinate sweeps (`Phylo2Vec-Full`) provide two-order-of-magnitude speedups over 64-thread CPU baselines up to $N = 10{,}000$, where quadratic device storage becomes limiting. Above this threshold, `Phylo2Vec-Minibatch` decouples the inner sweep cost from N, extending refinement to N = 1,000,000 and 10,000,000 leaves on a single GPU in minutes (Figures 2A and 3A). Streaming packed sequence data into device memory keeps host RAM below 0.4 GB up to N = 1,000,000 and near 2 GB at N = 10,000,000, whereas CPU baseline memory grows super-linearly with N (Figures 2C and 3C).

3. **Effect of sequence length on distance noise.** Increasing alignment length tenfold from L = 1,000 (`JC1k`) to L = 10,000 (`JC10k`) reduces the variance of pairwise distance

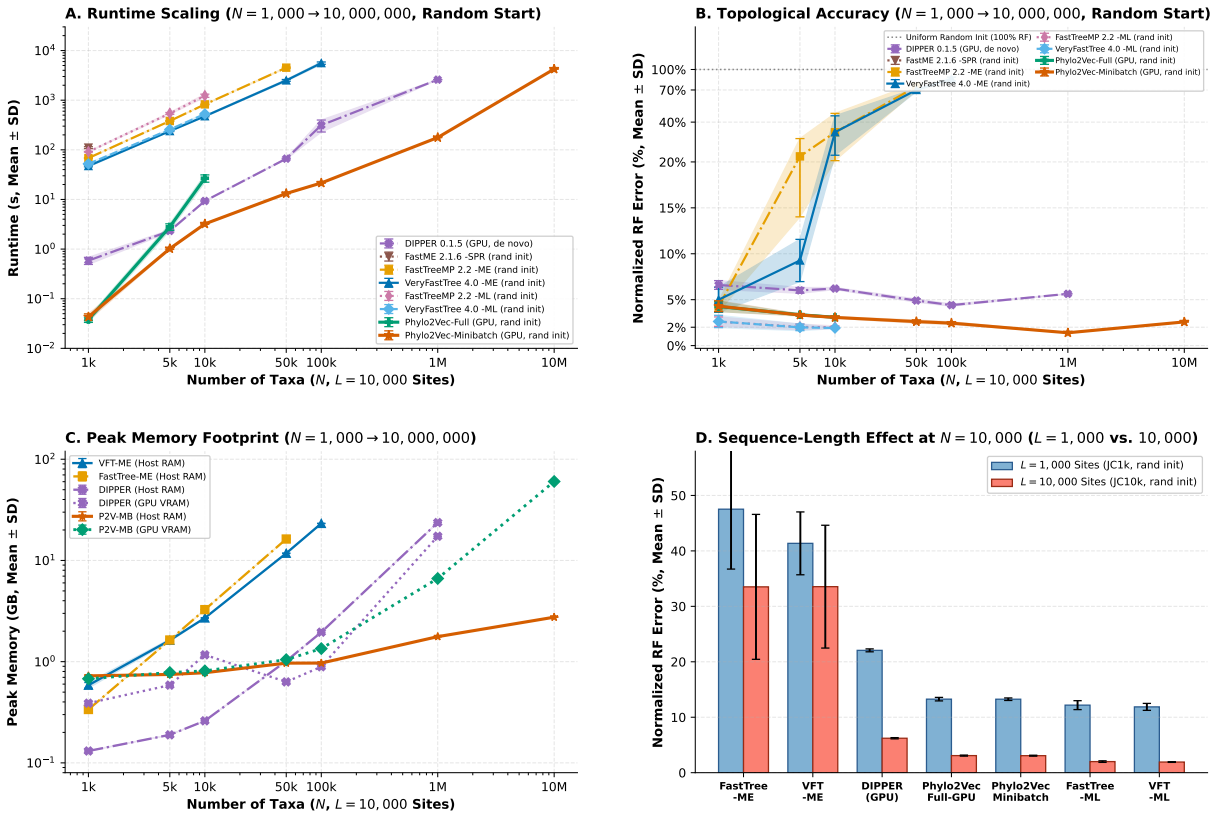


Figure 3: Sequence-to-tree refinement scaling on simulated **Jukes–Cantor** alignments across $N = 1{,}000$ to $10{,}000{,}000$ leaves ($R = 5$ independent replicates for $N \leq 100{,}000$; points and error bars show mean $\pm$ 1 SD). **(A)** Wall-clock refinement runtime on $L = 10{,}000$-site alignments (`JC10k`). **(B)** Normalized Robinson–Foulds topological error (%) on `JC10k`. **(C)** Peak host RAM and GPU VRAM footprint across $N = 1{,}000$ to $10{,}000{,}000$. **(D)** Effect of sequence length ($L = 1{,}000$ vs. $L = 10{,}000$ sites) on topological accuracy at $N = 10{,}000$ leaves.

estimates, lowering minimum-evolution RF error more than fourfold across all solvers and narrowing the absolute topological gap to full maximum-likelihood refinement while retaining a two-order-of-magnitude runtime advantage (Figure 3D).

5 Discussion

The method separates the size of a search block from the objective it evaluates. A skeleton can optimize connections among K components while retaining the contribution of every original leaf through balanced component distances. The same quadratic coordinate evaluator therefore supports both full-tree sweeps ($N \leq 10{,}000$) and restricted minibatch searches up to $N = 10{,}000{,}000$ leaves.

Across both the RNASim and Jukes–Cantor benchmarks, GPU full-tree and minibatch refinement achieve the same or slightly lower topological error and lower BME objective scores than 64-thread minimum-evolution baselines (`FastME`, `FastTree-ME`, and `VeryFastTree-ME`) and substantially lower topological error than unrefined GPU placement (`DIPPER`), while reducing refinement wall-clock time from hours to seconds or minutes. Streaming packed sequence data into device memory avoids large host allocations, enabling ten-million-leaf refinement with 10,000 sites on a single 141 GB H200 GPU using less than 2.2 GB of host RAM.

Three scope considerations apply to these results. First, the tested relabeling and block schedules are heuristic and do not guarantee global BME optimality, and the number of sweeps depends on the starting topology. Second, under finite alignment lengths, minimum evolution optimizes estimated pairwise distances rather than sequence likelihood; as shown in Figure 3D, longer alignments reduce distance estimation noise and narrow the gap between minimum-evolution and maximum-likelihood topologies. Third, for $N > 2{,}048$, the sequence minibatch implementation uses capped component profiles to accelerate inner skeleton proposals before verifying candidate blocks against complete leaf profiles in FP64 (Appendix B).

6 Conclusion

Shared coordinate evaluation and exact component reduction give a common basis for full-tree and block BME search. A quadratic sweep evaluates all valid coordinate updates, and the same computation on an exactly condensed skeleton preserves their full-tree objective changes. The resulting parallel algorithms let a search trade the scope of its candidate set against evaluation cost while retaining the specified BME objective.

References

[1] Y. Pauplin. Direct calculation of a tree length using a distance matrix. *Journal of Molecular Evolution*, 51(1):41–47, 2000.

[2] R. Desper and O. Gascuel. Fast and accurate phylogeny reconstruction algorithms based on the minimum-evolution principle. *Journal of Computational Biology*, 9(5):687–705, 2002.

[3] R. Desper and O. Gascuel. Theoretical foundation of the balanced minimum evolution method of phylogenetic inference and its relationship to weighted least-squares tree fitting. *Molecular Biology and Evolution*, 21(3):587–598, 2004. doi:10.1093/molbev/msh049.

[4] W. Hordijk and O. Gascuel. Improving the efficiency of SPR moves in phylogenetic tree search methods based on maximum likelihood. *Bioinformatics*, 21(24):4338–4347, 2005. doi:10.1093/bioinformatics/bti713.

[5] V. Lefort, R. Desper, and O. Gascuel. FastME 2.0: A comprehensive, accurate, and fast distance-based phylogeny inference program. *Molecular Biology and Evolution*, 32(10):2798–2800, 2015.

[6] M. J. Penn, N. Scheidwasser, M. P. Khurana, D. A. Duchêne, C. A. Donnelly, and S. Bhatt. Phylo2Vec: a vector representation for binary trees. *Systematic Biology*, 74(2):250–266, 2025. Advance access 2024. doi:10.1093/sysbio/syae030.

[7] M. J. Penn, N. Scheidwasser, J. Penn, C. A. Donnelly, D. A. Duchêne, and S. Bhatt. Leaping through tree space: continuous phylogenetic inference for rooted and unrooted trees. *Genome Biology and Evolution*, 15(12):evad213, 2023. doi:10.1093/gbe/evad213. Lemma numbers refer to the appendix of arXiv:2306.05739v4.

[8] C. Tapo and Q. Zhu. Parallel and scalable balanced minimum evolution algorithm for large-scale phylogenetic inference. *bioRxiv*, 2026. Preprint. doi:10.64898/2026.07.27.740982.

[9] F. J. Camerota Verdù, A. Gasparin, L. Bortolussi, and L. Castelli. Learning to explore tree neighbourhoods for phylogenetic inference. *Briefings in Bioinformatics*, 27(1):bbaf732, 2026. doi:10.1093/bib/bbaf732.

[10] D. C. Haws, T. L. Hodge, and R. Yoshida. Optimality of the neighbor joining algorithm and faces of the balanced minimum evolution polytope. *Bulletin of Mathematical Biology*, 73(11):2627–2648, 2011. doi:10.1007/s11538-011-9640-x.

[11] S. Mirarab, N. Nguyen, S. Guo, L.-S. Wang, J. Kim, and T. Warnow. PASTA: ultra-large multiple sequence alignment for nucleotide and amino-acid sequences. *Journal of Computational Biology*, 22(5):377–386, 2015. doi:10.1089/cmb.2014.0156.

[12] M. N. Price, P. S. Dehal, and A. P. Arkin. FastTree 2—approximately maximum-likelihood trees for large alignments. *PLOS ONE*, 5(3):e9490, 2010. doi:10.1371/journal.pone.0009490.

[13] C. Piñeiro, J. M. Abuín, and J. C. Pichel. VeryFastTree: speeding up the estimation of phylogenies for large alignments through parallelization and vectorization. *Bioinformatics*, 36(17):4658–4659, 2020. doi:10.1093/bioinformatics/btaa582.

[14] S. Walia, Z. Chen, Y.-H. Tseng, and Y. Turakhia. Ultrafast and ultralarge distance-based phylogenetics using DIPPER. *Nature Computational Science*, 2026. doi:10.1038/s43588-026-01015-8.

A Mathematical details

A.1 Block reduction

We use the component weights and attachment convention from Section 2.4.

Lemma A.1 (Normalization of leaf weights)**.** *For each component,* $\sum_{x \in S_a} \omega_a(x) = 1$.

Proof. A singleton has weight one. At a binary root, every leaf weight in each child component is multiplied by $1/2$. Induction over the component therefore gives total mass $\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1$. □

Proof of Theorem 2.2.

Proof. Separate the unordered leaf pairs into pairs within a component and pairs between components. For $x, y \in S_a$, the path stays inside S_a and is unchanged by rearranging the skeleton. The sum of all such terms is

$$C = \sum_a \sum_{\substack{x<y \\ x,y \in S_a}} D_{xy}\, 2^{-q_{T_G}(x,y)},$$

which is independent of G.

For $x \in S_a$ and $y \in S_b$ with $a \neq b$, the path contains the branching vertices inside S_a, those on the skeleton path, and those inside S_b. These sets of vertices are disjoint, giving

$$q_{T_G}(x, y) = h_a(x) + q_G(a, b) + h_b(y).$$

Therefore its weight factors as $2^{-q_{T_G}(x,y)} = \omega_a(x)\, 2^{-q_G(a,b)}\, \omega_b(y)$. Summing between-component terms gives

$$\begin{aligned} \sum_{a<b} \sum_{\substack{x \in S_a \\ y \in S_b}} D_{xy}\, 2^{-q_{T_G}(x,y)} &= \sum_{a<b} 2^{-q_G(a,b)} \sum_{\substack{x \in S_a \\ y \in S_b}} \omega_a(x)\omega_b(y) D_{xy} \\ &= L(G; \widetilde{D}). \end{aligned}$$

Adding the within-component constant proves Equation (8). Subtracting two such decompositions cancels C and proves Equation (9). □

A.2 Ordered representation

For an ordered vector, initialize $r[x] = x$ and process $i = N - 2, \ldots, 0$. Join components $r[v_i]$ and $r[i+1]$ at node $2N - 2 - i$, then replace $r[v_i]$ by this node. The active components have distinct minimum labels $0, \ldots, i+1$ before each merge, giving an $O(N)$ decoder.

Relabeling uses Penn et al.'s linear-time construction [7, Lemmas 0.9–0.11]. Starting with root label zero and counter $t = 1$, visit internal nodes parent before child. Each passes its label to one child, labels the other t, records the inherited label as v_{t-1}, and increments t. Our first orientation lets the child with smaller minimum leaf index inherit; the second reverses that choice. Both preserve the tree and its BME score when both axes of D are permuted by the inverse leaf-label map. The construction guarantees representational coverage after relabeling; optimizer reachability depends on its search policy. For unrooted tree-to-vector conversion, insert the artificial root on the edge incident to leaf zero.

A.3 Coordinate replacements are SPR moves

For an ordered v and a leaf $\ell \geq i + 1$, follow the chain $\ell_0 = \ell$, $\ell_{s+1} = v_{\ell_s - 1}$ while $\ell_s > i + 1$. Each step uses a coordinate with index greater than i, and the chain strictly decreases. Let P_i be the set of leaves whose chain stops at $i + 1$; it contains $i + 1$ and does not depend on v_i.

Lemma A.2 (Coordinate neighbors are SPR neighbors)**.** *Let $v \in \mathcal{V}_N^{\mathrm{cf}}$, $0 \le i \le N-2$, and $0 \le j \le 2i$. Then $T(v^{(i\leftarrow j)})$ either equals $T(v)$ or is obtained from $T(v)$ by one SPR move that prunes the subtree on the leaves P_i.*

Proof. Read $w = v^{(i\leftarrow j)}$ by leaf addition [6]. Starting from leaf 0, coordinate w_k inserts leaf $k+1$ on one of the $2k+1$ edges of the current rooted tree on leaves $0,\dots,k$, including the edge above its root, and a value $w_k \le k$ selects the pendant edge of leaf w_k. Only step i depends on j. It inserts leaf $i+1$ on some edge, creating a vertex p. Every later step $k > i$ has $w_k = v_k \le k$ and inserts leaf $k+1$ on the pendant edge of leaf v_k. The new leaf therefore lies on the same side of p as leaf v_k. By induction on k, the leaves on the side of p that contains $i+1$ are exactly P_i. The insertions among these leaves build a subtree whose topology does not depend on j. Removing P_i and suppressing p leaves the tree T^- produced by steps $0,\dots,i-1$ and the insertions of leaves outside P_i, which also does not depend on j. Thus every $T(w)$ is T^- with the fixed subtree on P_i attached at one edge of the unrooted T^-. If leaf $i+1$ is inserted above the root, that edge is the one through the root of T^-, which becomes a single edge when the artificial root is suppressed. Taking $j = v_i$ recovers $T(v)$. Two such trees differ only in the attachment edge of P_i, so they are equal or related by one SPR move. □

The lemma bounds the coordinate neighborhood by the SPR neighborhood. It does not show the reverse inclusion. The mapping below identifies the attachment edge for each j.

A.4 Mapping coordinate values to regraft edges

The ordered decoder numbers leaves $0,\dots,N-1$ and internal nodes $N,\dots,2N-2$. The last index is the artificial root. Suppressing it leaves a computational root $R = 2N-3$ and the top edge joining R to the artificial root's other child, denoted c_{other}. Index ordinary edges by their child endpoints.

For row i, let $E_i[u]$ be the score change from pruning the component associated with leaf $i+1$ and regrafting it on the edge indexed by u. The half-tree SPR traversal supplies these values. Let $b_i(j)$ be the root of the component containing leaf j after the merges with coordinate index greater than i. Then the edge-to-coordinate map is

$$\Delta_{i,j} = \begin{cases} E_i[b_i(j)], & 0 \le j \le i, \\ E_i[(2N-2) - 2i + j], & i+1 \le j \le 2i-2, \\ E_i[c_{\mathrm{other}}], & j \in \{2i-1, 2i\} \cap [i+1, 2i]. \end{cases} \tag{13}$$

The first case identifies the component on the corresponding pendant edge of the prefix tree. In the ordered decoder's node numbering, the remaining non-root prefix edges occupy consecutive indices $2N-1-i,\dots,2N-4$, giving the affine map in the second case. The last two rooted edge choices collapse onto the same unrooted top edge. At $i = 1$, the middle case is empty and $j = 1$ belongs to the first case. Only $j = 2$ takes the collapsed-root value. Coordinate zero has no nontrivial alternative.

On a CPU, the array b_i can be maintained incrementally as the merge order advances. On a GPU, independent rows reconstruct their mappings using the latest eligible merge for each minimum leaf. When several merges have the same minimum leaf, only the latest eligible merge may write that destination. This avoids concurrent writes that would otherwise depend on thread scheduling.

A.5 Parallel evaluation and reduced storage

After initialization and the required transpose, Equation (4) has independent columns. For a non-root ancestor u of w, let $\mathrm{sib}_u(w)$ be the child of u whose subtree excludes w. The complementary

direction is computed from the root down:

$$A(u, w) = \tfrac{1}{2}\{A(\mathrm{sib}_u(w), w) + A(\mathrm{parent}(u), w)\}. \tag{14}$$

The degree-three root is initialized using its two branches away from w.

For reduced storage, retain only internal-to-leaf distances. These initialize the leaf entries of an internal row w; a leaf row instead starts from D. Bottom-up averaging fills internal entries, and Equation (14) overwrites ancestors of w after the root initialization. Set the diagonal to zero. For a vertex u in a disjoint component, the bottom-up calculation follows directly by linearity. For a descendant u of w, the initial leaf entries already use the component at w facing away from u, so the same averaging is valid. Ancestors require the complementary direction supplied by the overwrite. Thus each row costs $O(N + \mathrm{depth}(w)) = O(N)$ to reconstruct. A constant number of reconstructions per pruned component retains $O(N^2)$ sweep work while reducing the balanced-distance table from approximately $4N^2$ to N^2 entries.

B Search settings

Algorithms 1 and 2 use the following settings, with scores evaluated in the recorded component labeling. The CPU solver uses sweep budget $I = 500$, escape budget $E = 2$, and $\eta = 10^{-12}(|\ell| + 1)$ at score ℓ.

Block construction. For requested size $3 \le K_\star < N$, expose w consecutive leaves in the current depth-first order, with $w = \min\{K_\star - 2, \max\{4, \lfloor 2K_\star/3 \rfloor\}\}$. Block b starts at $b \max\{1, \lfloor w/2 \rfloor\} \bmod N$. Condense maximal inactive subtrees, then split the largest such component until the skeleton reaches $K_\star$ leaves or no split remains possible. The resulting size K can differ from the target $K_\star$. Rebuild the leaf order, components, and outside information from the current tree for every block. Hierarchical schedules combine backbone and local windows with staggered or larger windows crossing component boundaries. Full-tree search uses $K = N$ singleton components.

Row selection and joint acceptance. RankMin selects $j_i \in \arg\min_{0 \le j \le 2i} \Delta_{i,j}$, retains changes below $-\eta$, and sorts them increasingly, resolving ties by deterministic traversal and row order. Let $d_{\min} < 0$ be the best single change. BatchStep greedily packs at most eight rows, excluding shared marked pruned-subtree nodes and attachment endpoints. After the first row, require $\Delta_{i,j_i} < 0.05 d_{\min}$, and test a batch only if the sum is below $1.05 d_{\min}$. Each trial starts from the same pre-trial encoding v and is accepted only if its complete score satisfies $L(T(v'); M_\pi) < \ell + d_{\min} - \eta$. On rejection, retry successively halved ranked prefixes down to two rows; if none passes, use the best single replacement. The predicted sum only screens candidates.

Completion and escapes. Decode restores component identities and suppresses the artificial root. ExtraSPR and RemainingSPR return an encoding in the current labeling or $\perp$ if no change is below $-\eta$. After an unsuccessful completion scan, an escape is permitted only if a subsequent sweep remains. For each row, retain the smallest change in $(\eta, 0.05)$ and sort these increasingly. Attempt $e \in \{1, 2\}$ combines the first and $(e+1)$th candidates if both exist and their sum is below 0.02; otherwise it returns $\perp$. The thresholds are in objective units. Escapes can increase the working score, so the solver returns the best scored tree, including the final trial.

GPU policy. The GPU solver packs SPR moves with disjoint marked paths, rescores each combined trial on the next sweep, and retains the best scored skeleton over a bounded number of sweeps. It tries reversed labeling when no improving row remains, without CPU batch-halving or full SPR completion. The best trial is checked against the entry skeleton before committing a block. Intermediate trial scores can increase. For sequence refinement above 2,048 leaves, inner skeleton proposals cap component leaf-weight lists at 64 entries; before committing a block, affected

condensed pairs are recomputed across all leaf contributions and checked with Equation (11) in FP64 using acceptance margin 10^{-6}.

C Additional experimental results

C.1 Sequence-refinement scaling tables

Tables 1, 2, and 3 provide the complete numerical measurements corresponding to Figures 2 and 3 on simulated RNASim ($L = 1{,}596$) and Jukes–Cantor ($L = 10{,}000$ and $1{,}000$) alignments.

Table 1: Sequence-to-tree scaling on simulated **RNASim** alignments ($L = 1{,}596$ sites) from $N = 1{,}000$ to $10{,}000{,}000$ leaves starting from uniform random trees (100.0% initial RF), alongside `DIPPER 0.1.5` (*de novo*). Entries for $N \leq 100{,}000$ report mean ± SD over $R = 5$ independent replicates.

N	R	Method	RF (%)	Time (s)	ΔBME	Host RAM (MB)	VRAM (MB)	Hardware
1,000	5	Uniform Random Init	99.97 ± 0.04	0.00 ± 0.00	0.00 ± 0.00	—	—	—
1,000	5	DIPPER 0.1.5 (GPU, *de novo*)	20.98 ± 0.82	0.47 ± 0.07	—	121.6 ± 0.5	392.0	64 CPU + 1× A6000
1,000	5	FastTreeMP 2.2 (`-noml -intree`)	19.70 ± 3.12	11.94 ± 0.20	—	133.5 ± 2.9	—	64 CPU
1,000	5	VeryFastTree 4.0 (`-noml -intree`)	18.62 ± 0.89	8.04 ± 0.24	—	131.6 ± 2.2	—	64 CPU
1,000	5	FastTreeMP 2.2 (ME+ML `-intree`)	**14.42 ± 0.47**	18.30 ± 0.58	—	133.5 ± 2.9	—	64 CPU
1,000	5	VeryFastTree 4.0 (ME+ML `-intree`)	14.62 ± 0.65	9.42 ± 0.27	—	133.5 ± 2.9	—	64 CPU
1,000	5	**Phylo2Vec-Full**	18.13 ± 0.79	**0.04 ± 0.01**	**−144.10 ± 1.46**	106.8 ± 0.0	688.8	64 CPU + 1× H100
1,000	5	**Phylo2Vec-Minibatch**	18.31 ± 1.19	**0.04 ± 0.00**	−144.10 ± 1.45	735.7 ± 1.4	690.8	64 CPU + 1× H100
5,000	5	Uniform Random Init	100.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	—	—	—
5,000	5	DIPPER 0.1.5 (GPU, *de novo*)	18.82 ± 0.77	2.07 ± 0.06	—	133.0 ± 0.0	580.0	64 CPU + 1× A6000
5,000	5	FastTreeMP 2.2 (`-noml -intree`)	45.55 ± 9.45	70.38 ± 1.91	—	290.4 ± 0.5	—	64 CPU
5,000	5	VeryFastTree 4.0 (`-noml -intree`)	43.17 ± 9.67	42.96 ± 0.86	—	317.6 ± 7.4	—	64 CPU
5,000	5	FastTreeMP 2.2 (ME+ML `-intree`)	**13.62 ± 1.33**	117.48 ± 5.45	—	292.2 ± 1.8	—	64 CPU
5,000	5	VeryFastTree 4.0 (ME+ML `-intree`)	14.85 ± 1.41	50.22 ± 1.55	—	318.6 ± 8.0	—	64 CPU
5,000	5	**Phylo2Vec-Full**	14.94 ± 0.61	2.29 ± 0.16	**−869.75 ± 6.90**	111.7 ± 0.7	694.8	64 CPU + 1× H100
5,000	5	**Phylo2Vec-Minibatch**	14.80 ± 0.62	**0.49 ± 0.05**	−869.75 ± 6.89	739.9 ± 0.7	696.8	64 CPU + 8× H100
10,000	5	Uniform Random Init	100.00 ± 0.00	0.01 ± 0.01	0.00 ± 0.00	—	—	—
10,000	5	DIPPER 0.1.5 (GPU, *de novo*)	17.78 ± 0.38	8.90 ± 0.10	—	145.8 ± 0.8	1156.0	64 CPU + 1× A6000
10,000	5	FastTreeMP 2.2 (`-noml -intree`)	64.34 ± 7.47	155.42 ± 5.57	—	570.1 ± 1.3	—	64 CPU
10,000	5	VeryFastTree 4.0 (`-noml -intree`)	58.97 ± 6.78	90.56 ± 3.09	—	542.3 ± 7.1	—	64 CPU
10,000	5	FastTreeMP 2.2 (ME+ML `-intree`)	14.47 ± 1.33	280.93 ± 11.76	—	571.8 ± 1.3	—	64 CPU
10,000	5	VeryFastTree 4.0 (ME+ML `-intree`)	14.29 ± 0.59	103.90 ± 3.22	—	545.0 ± 6.8	—	64 CPU
10,000	5	**Phylo2Vec-Full**	13.69 ± 0.36	23.00 ± 3.28	**−1868.38 ± 10.02**	429.9 ± 157.2	698.8	64 CPU + 1× H100
10,000	5	**Phylo2Vec-Minibatch**	**13.65 ± 0.40**	**1.39 ± 0.66**	−1868.34 ± 9.98	743.8 ± 1.1	700.8	64 CPU + 8× H100
50,000	5	Uniform Random Init	100.00 ± 0.00	0.04 ± 0.00	0.00 ± 0.00	—	—	—
50,000	5	DIPPER 0.1.5 (GPU, *de novo*)	19.50 ± 0.15	59.83 ± 5.51	—	246.6 ± 0.8	446.0	64 CPU + 1× A6000
50,000	5	FastTreeMP 2.2 (`-noml -intree`)	89.30 ± 1.13	874.43 ± 24.09	—	2835.9 ± 0.6	—	64 CPU
50,000	5	VeryFastTree 4.0 (`-noml -intree`)	88.09 ± 1.29	486.95 ± 38.28	—	2405.4 ± 15.5	—	64 CPU
50,000	5	**Phylo2Vec-Minibatch**	**9.91 ± 0.15**	**3.71 ± 0.19**	**−10943.39 ± 12.09**	782.2 ± 1.1	746.8	64 CPU + 8× H100
100,000	5	Uniform Random Init	100.00 ± 0.00	0.10 ± 0.00	0.00 ± 0.00	—	—	—
100,000	5	DIPPER 0.1.5 (GPU, *de novo*)	16.80 ± 0.19	216.26 ± 0.22	—	386.5 ± 2.8	512.0	64 CPU + 1× A6000
100,000	5	FastTreeMP 2.2 (`-noml -intree`)	92.62 ± 0.59	1853.11 ± 67.03	—	5677.2 ± 2.3	—	64 CPU
100,000	5	VeryFastTree 4.0 (`-noml -intree`)	92.73 ± 1.37	1024.52 ± 37.34	—	4651.7 ± 24.7	—	64 CPU
100,000	5	**Phylo2Vec-Minibatch**	**8.76 ± 0.05**	**6.75 ± 0.30**	**−23262.65 ± 20.42**	830.2 ± 0.9	794.8	64 CPU + 8× H100
1,000,000	1	Uniform Random Init	100.00	0.00	0.00	—	—	—
1,000,000	1	DIPPER 0.1.5 (GPU, *de novo*)	11.25	978.23	—	4189.6	1871.8	64 CPU + 1× A6000
1,000,000	1	**Phylo2Vec-Minibatch**	**1.17**	**69.01**	**−306164.97**	**1043.5**	**1656.8**	64 CPU + 8× H100
10,000,000	1	Uniform Random Init	100.00	0.00	0.00	—	—	—
10,000,000	1	**Phylo2Vec-Minibatch**	**4.13**	**974.61**	**−3319791.02**	**2575.1**	10240.8	64 CPU + 8× H100

Table 2: Sequence-to-tree scaling on simulated Jukes–Cantor alignments with $L = 10{,}000$ sites (`JC10k`) from $N = 1{,}000$ to 10,000,000 leaves starting from uniform random trees (100.0% initial RF), alongside `DIPPER 0.1.5` (*de novo*). Entries for $N \leq 100{,}000$ report mean ± SD over $R = 5$ independent replicates.

N	R	Method	RF (%)	Time (s)	ΔBME	Host RAM (MB)	VRAM (MB)	Hardware
1,000	5	Uniform Random Init	99.97 ± 0.04	0.00 ± 0.00	0.00 ± 0.00	—	—	—
1,000	5	DIPPER 0.1.5 (GPU, *de novo*)	6.60 ± 0.48	0.59 ± 0.10	—	134.2 ± 1.0	398.0	64 CPU + 1× A6000
1,000	5	FastME 2.1.6 (`-s -u`)	4.25 ± 0.57	110.76 ± 4.85	—	138.0 ± 2.5	—	64 CPU
1,000	5	FastTreeMP 2.2 (`-noml -intree`)	4.25 ± 0.57	68.20 ± 1.47	—	345.0 ± 0.6	—	64 CPU
1,000	5	VeryFastTree 4.0 (`-noml -intree`)	4.99 ± 1.36	47.15 ± 1.77	—	599.4 ± 37.4	—	64 CPU
1,000	5	FastTreeMP 2.2 (ME+ML `-intree`)	**2.61 ± 0.52**	91.23 ± 1.30	—	395.4 ± 3.9	—	64 CPU
1,000	5	VeryFastTree 4.0 (ME+ML `-intree`)	2.63 ± 0.67	52.11 ± 1.45	—	603.1 ± 44.1	—	64 CPU
1,000	5	**Phylo2Vec-Full**	4.25 ± 0.56	**0.04 ± 0.00**	**−132.79 ± 1.63**	112.0 ± 0.0	696.8	64 CPU + 1× H100
1,000	5	**Phylo2Vec-Minibatch**	4.29 ± 0.58	**0.04 ± 0.01**	**−132.79 ± 1.63**	741.0 ± 0.9	698.8	64 CPU + 1× H100
5,000	5	Uniform Random Init	100.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	—	—	—
5,000	5	DIPPER 0.1.5 (GPU, *de novo*)	6.01 ± 0.31	2.32 ± 0.10	—	193.8 ± 0.1	600.0	64 CPU + 1× A6000
5,000	5	FastTreeMP 2.2 (`-noml -intree`)	22.02 ± 8.00	380.93 ± 5.04	—	1676.2 ± 3.6	—	64 CPU
5,000	5	VeryFastTree 4.0 (`-noml -intree`)	9.27 ± 2.30	238.48 ± 5.14	—	1666.2 ± 47.3	—	64 CPU
5,000	5	FastTreeMP 2.2 (ME+ML `-intree`)	2.04 ± 0.31	541.68 ± 22.96	—	1742.7 ± 4.4	—	64 CPU
5,000	5	VeryFastTree 4.0 (ME+ML `-intree`)	**1.98 ± 0.36**	256.49 ± 5.46	—	1649.8 ± 45.5	—	64 CPU
5,000	5	**Phylo2Vec-Full**	3.36 ± 0.07	2.81 ± 0.42	**−738.35 ± 9.75**	136.5 ± 0.0	720.8	64 CPU + 1× H100
5,000	5	**Phylo2Vec-Minibatch**	3.33 ± 0.12	**1.02 ± 0.04**	−738.28 ± 9.79	765.5 ± 0.9	722.8	64 CPU + 8× H100
10,000	5	Uniform Random Init	100.00 ± 0.00	0.01 ± 0.01	0.00 ± 0.00	—	—	—
10,000	5	DIPPER 0.1.5 (GPU, *de novo*)	6.21 ± 0.11	9.31 ± 0.05	—	266.6 ± 0.8	1196.0	64 CPU + 1× A6000
10,000	5	FastTreeMP 2.2 (`-noml -intree`)	33.51 ± 13.07	823.06 ± 11.99	—	3345.3 ± 9.4	—	64 CPU
10,000	5	VeryFastTree 4.0 (`-noml -intree`)	33.55 ± 11.07	475.03 ± 13.05	—	2766.5 ± 14.1	—	64 CPU
10,000	5	FastTreeMP 2.2 (ME+ML `-intree`)	2.00 ± 0.14	1230.72 ± 91.55	—	3411.9 ± 9.2	—	64 CPU
10,000	5	VeryFastTree 4.0 (ME+ML `-intree`)	**1.92 ± 0.06**	525.70 ± 20.07	—	2770.2 ± 26.4	—	64 CPU
10,000	5	**Phylo2Vec-Full**	3.08 ± 0.09	26.76 ± 4.50	**−1542.81 ± 15.45**	440.4 ± 136.2	750.8	64 CPU + 1× H100
10,000	5	**Phylo2Vec-Minibatch**	3.07 ± 0.08	**3.22 ± 0.06**	−1542.67 ± 15.40	796.2 ± 0.0	752.8	64 CPU + 8× H100
50,000	5	Uniform Random Init	100.00 ± 0.00	0.04 ± 0.00	0.00 ± 0.00	—	—	—
50,000	5	DIPPER 0.1.5 (GPU, *de novo*)	4.90 ± 0.17	66.30 ± 1.36	—	1038.6 ± 6.4	646.0	64 CPU + 1× A6000
50,000	4	FastTreeMP 2.2 (`-noml -intree`)	74.25 ± 4.51	4525.52 ± 115.67	—	16673.0 ± 19.7	—	64 CPU
50,000	5	VeryFastTree 4.0 (`-noml -intree`)	70.03 ± 2.04	2506.54 ± 103.01	—	12062.2 ± 61.7	—	64 CPU
50,000	5	**Phylo2Vec-Minibatch**	**2.61 ± 0.17**	**13.19 ± 0.21**	**−8408.27 ± 56.33**	**990.5 ± 1.6**	1004.8	64 CPU + 8× H100
100,000	5	Uniform Random Init	100.00 ± 0.00	0.10 ± 0.00	0.00 ± 0.00	—	—	—
100,000	5	DIPPER 0.1.5 (GPU, *de novo*)	4.41 ± 0.12	313.35 ± 84.78	—	2001.4 ± 3.3	912.0	64 CPU + 1× A6000
100,000	5	VeryFastTree 4.0 (`-noml -intree`)	84.88 ± 2.64	5588.08 ± 300.03	—	23767.2 ± 110.4	—	64 CPU
100,000	5	**Phylo2Vec-Minibatch**	**2.45 ± 0.15**	**21.48 ± 0.09**	**−17306.92 ± 122.36**	**991.5 ± 1.4**	1310.8	64 CPU + 8× H100
1,000,000	1	Uniform Random Init	100.00	0.00	0.00	—	—	—
1,000,000	1	DIPPER 0.1.5 (GPU, *de novo*)	5.64	2592.75	—	24225.3	17747.6	64 CPU + 1× A6000
1,000,000	1	**Phylo2Vec-Minibatch**	**1.40**	**176.97**	**−201262.38**	**1809.1**	**6806.8**	64 CPU + 8× H100
10,000,000	1	Uniform Random Init	100.00	0.00	0.00	—	—	—
10,000,000	1	**Phylo2Vec-Minibatch**	**2.58**	**4240.82**	**−2232357.84**	**2816.2**	61726.8	64 CPU + 8× H100

Table 3: Sequence-length sensitivity at $N = 10{,}000$ leaves starting from uniform random trees (100.0% initial RF): simulated Jukes–Cantor alignments with $L = 1{,}000$ sites (`JC1k`) vs. $L = 10{,}000$ sites (`JC10k`) over $R = 5$ independent replicates (mean ± SD).

Method	$L = 1{,}000$ sites (`JC1k`, $R = 5$)			$L = 10{,}000$ sites (`JC10k`, $R = 5$)		
	Norm. RF (%)	Time (s)	ΔBME	Norm. RF (%)	Time (s)	ΔBM
Uniform Random Init	100.00 ± 0.00	0.01 ± 0.01	0.00 ± 0.00	100.00 ± 0.00	0.01 ± 0.01	0.00 ± 0.
DIPPER 0.1.5 (GPU, *de novo*)	22.06 ± 0.26	8.74 ± 0.08	—	6.21 ± 0.11	9.31 ± 0.05	
FastTreeMP 2.2 (`-noml -intree`)	47.52 ± 10.80	85.00 ± 3.06	—	33.51 ± 13.07	823.06 ± 11.99	
VeryFastTree 4.0 (`-noml -intree`)	41.36 ± 5.67	50.23 ± 2.15	—	33.55 ± 11.07	475.03 ± 13.05	
FastTreeMP 2.2 (ME+ML `-intree`)	12.18 ± 0.80	129.78 ± 5.78	—	2.00 ± 0.14	1230.72 ± 91.55	
VeryFastTree 4.0 (ME+ML `-intree`)	**11.87 ± 0.63**	55.72 ± 2.27	—	**1.92 ± 0.06**	525.70 ± 20.07	
Phylo2Vec-Full	13.26 ± 0.32	28.63 ± 3.34	**−1561.93 ± 26.97**	3.08 ± 0.09	26.76 ± 4.50	**−1542.81 ± 15.**
Phylo2Vec-Minibatch	13.25 ± 0.23	**0.90 ± 0.03**	−1561.84 ± 26.90	3.07 ± 0.08	**3.22 ± 0.06**	−1542.67 ± 15.

C.2 Numerical verification and small-tree CPU optimization

Table 4 compares coordinate and condensed changes with independent direct BME calculations. Tables 5 and 6 report single-core CPU optimization comparisons against the in-memory FastME BME-SPR reference (FME) on Apple M5 across additive and noisy distance matrices ($N = 20$ to 200).

Table 4: Agreement with independent BME calculations. Errors are maximum absolute discrepancies over the stated tests.

Calculation	Test scope	Maximum error
CPU coordinate differences	36,000 changes, $N = 20$	3.553×10^{-14}
Ordered-tree enumeration	28,488 changes, $N = 4$–7	2.776×10^{-15}
Relabeling, first orientation	200 trees, $N = 25$	1.776×10^{-14}
Relabeling, reversed orientation	Same 200 trees	2.665×10^{-14}
Condensed SPR differences	16,231 changes, $N = 40$, $K = 16$	1.564×10^{-13}
FP32 CUDA gradient	Reported sizes $N = 100$–4,000	1.11×10^{-6}

Table 5: Additive distance comparison on Apple M5.

Objective and topology

N	FME L	P2V L	Win/tie/loss	FME RF	P2V RF
20	11.12739	11.12739	**0 / 20 / 0**	0.0000	0.0000
50	29.42572	29.42572	**0 / 20 / 0**	0.0000	0.0000
100	59.52693	59.52693	**0 / 10 / 0**	0.0000	0.0000
150	89.04365	89.04365	**0 / 5 / 0**	0.0000	0.0000
200	116.32398	116.32398	**0 / 5 / 0**	0.0000	0.0000

Computational cost

N	FME iters.	P2V evals.	FME ms	P2V ms	P2V med. [IQR]	Ratio
20	13.4	11.2	0.12 ± 0.02	0.08 ± 0.01	0.07 [0.02]	**1.54**×
50	42.2	19.4	1.58 ± 0.35	0.57 ± 0.13	0.59 [0.16]	**2.78**×
100	97.7	35.4	9.05 ± 0.84	2.63 ± 0.26	2.65 [0.44]	**3.44**×
150	156.2	49.4	31.47 ± 0.77	7.92 ± 0.86	7.75 [0.64]	**3.97**×
200	211.0	63.6	75.46 ± 2.42	18.14 ± 0.74	18.05 [1.17]	**4.16**×

Table 6: Noisy distance comparison on Apple M5.

Objective and topology

N	FME L	P2V L	Win/tie/loss	FME RF	P2V RF
20	11.19777	11.19777	**0 / 20 / 0**	0.0500	0.0500
50	29.25507	29.25607	**1 / 18 / 1**	0.0660	0.0649
100	59.25471	59.25471	**0 / 10 / 0**	0.0526	0.0526
150	92.14846	92.14846	**0 / 5 / 0**	0.0395	0.0395
200	120.94220	120.94220	**1 / 4 / 0**	0.0467	0.0487

Computational cost

N	FME iters.	P2V evals.	FME ms	P2V ms	P2V med. [IQR]	Ratio
20	14.4	12.4	0.07 ± 0.01	0.05 ± 0.01	0.04 [0.02]	**1.60**×
50	46.1	25.9	1.07 ± 0.07	0.49 ± 0.09	0.49 [0.10]	**2.18**×
100	102.6	41.0	10.62 ± 2.53	3.32 ± 0.56	3.18 [0.52]	**3.19**×
150	166.0	61.0	33.97 ± 1.01	10.15 ± 1.35	10.17 [1.35]	**3.35**×
200	235.8	84.4	85.16 ± 2.06	24.67 ± 2.79	24.51 [4.48]	**3.45**×

C.3 GPU evaluation

Tables 7–10 report coordinate-sweep and condensation costs at fixed trees. Single-GPU speedups use different host CPU references. In Table 10, $2N^2$ is a rectangular-workload proxy; the number of valid coordinate entries is $(N-1)^2$. Memory units follow the benchmark records.

For Table 8, each skeleton contains 15,000 expanded leaves and 5,000 condensed components. Epoch estimates multiply mean condensation-plus-sweep time by $\lceil N/15{,}000 \rceil$ and exclude additional search work and optimization to convergence. Validation discrepancies are observed errors on the benchmark checks.

Table 7: Single-GPU gradient timings and agreement with the CPU reference. Times are milliseconds.

RTX A6000 timings and reported maximum errors

N	A6000 full	Ratio	A6000 tiled	Max. error
100	0.061	**8.44**×	0.078	3.68e-07
250	0.186	**15.92**×	0.207	6.22e-07
500	0.382	**32.95**×	0.511	6.48e-07
1,000	1.035	**33.69**×	1.988	9.29e-07
2,000	3.851	**40.15**×	7.288	1.11e-06
4,000	11.053	**62.48**×	43.124	1.07e-06

H100 timings

N	CPU ref.	H100 avg.	H100 SPR	H100 total	Ratio	H100 tiled
100	0.249	0.044	0.027	0.071	**3.51**×	0.095
250	1.964	0.116	0.048	0.164	**11.98**×	0.205
500	8.176	0.227	0.077	0.304	**26.89**×	0.460
1,000	32.336	0.661	0.105	0.766	**42.21**×	1.030
2,000	154.091	1.557	0.726	2.283	**67.50**×	4.273
4,000	701.375	3.565	2.654	6.219	**112.78**×	22.327

The maximum-error column is the reported combined check across the GPU paths. All listed checks pass their benchmark tolerance. CPU baselines differ between hosts.

Table 8: Embedding-based condensed sweeps with $K = 20{,}000$ skeleton leaves. The epoch columns are batch-count times mean condensation-plus-gradient time.

Two A6000 GPUs

N	Batches	2 A6000 cond.+grad. (ms)	Epoch (s)
1,000,000	67	4.45 + 284.44	19.36
5,000,000	334	8.42 + 283.02	97.34
10,000,000	667	17.29 + 281.58	199.35

Eight H100 GPUs

N	Cond. ms	Grad. ms	Epoch s	MB/GPU	Check
1,000,000	2.69	48.53	**3.43**	7,793	**PASS** (1.4e-05)
5,000,000	4.86	48.48	**17.82**	7,824	**PASS** (2.1e-06)
10,000,000	10.20	48.57	**39.20**	7,862	**PASS** (2.2e-06)

This workload uses a bilinear embedding distance and is separate from sequence-distance refinement. The validation figures are observed discrepancies, not universal error bounds.

Table 9: Full-table and tiled multi-GPU sweeps. Times are milliseconds and memory is reported in MB.

A6000 timings and memory columns

N	2 A6000 full	2 A6000 tiled	Full memory	Tiled memory
1,000	0.93	5.75	27	39
2,000	4.89	13.90	92	93
4,000	14.69	56.85	336	247
10,000	90.45	249.59	1,985	1,076
20,000	422.73	1,212.27	7,785	3,677
40,000	*OOM*	5,628.62	31,142	13,614

Full-table H100 sweep

N	1 H100 [avg.+SPR]	8 H100	Ratio
1,000	0.76 [0.65 + 0.11]	0.79	0.96×
2,000	2.21 [1.47 + 0.74]	1.71	**1.29**×
4,000	6.01 [3.57 + 2.44]	3.98	**1.51**×
10,000	29.61 [10.10 + 19.50]	13.03	**2.27**×
20,000	155.93 [23.23 + 132.70]	41.06	**3.80**×
40,000	816.34 [60.08 + 756.26]	**158.36**	**5.15**×

Tiled H100 sweep

N	1 H100 tiled	8 H100 tiled	Top-1 check
1,000	1.46	4.47	**PASS** (0.00e+00)
2,000	6.14	5.21	**PASS** (2.38e-07)
4,000	30.78	10.98	**PASS** (0.00e+00)
10,000	250.69	57.53	**PASS** (0.00e+00)
20,000	1,123.43	244.13	**PASS** (4.77e-07)
40,000	4,768.31	**1,032.83**	**PASS** (0.00e+00)

Table 10: Distributed sweeps on eight H100 GPUs.

Timing

N	Work proxy $2N^2$	Distance ms	SPR ms	Total s
40,000	3.20×10^9	514.54	517.92	**1.032**
80,000	1.28×10^{10}	2,112.82	2,321.28	**4.434**
120,000	2.88×10^{10}	4,785.75	4,803.60	**9.589**
180,000	6.48×10^{10}	10,848.67	13,977.73	**24.826**
250,000	1.25×10^{11}	39,981.55	43,721.48	**83.703**
350,000	2.45×10^{11}	76,240.05	96,515.83	**172.756**
380,000	2.89×10^{11}	89,992.00	115,457.57	**205.450**

Memory and execution mode

N	MB/GPU	Aggregate GB	Mode and check
40,000	2,386	18.6	`Full-Shard` [**PASS**]
80,000	7,824	61.1	`Shard+Calib` [**PASS**]
120,000	16,314	127.5	`Shard+Calib` [**PASS**]
180,000	34,770	271.6	`Shard+Calib` [**PASS**]
250,000	62,296	486.7	`Shard+Calib` [**PASS**]
350,000	62,180	485.8	`StreamD+Calib` [**PASS**]
380,000	72,946	569.9	`StreamD+Calib` [**PASS**]

At $N = 400{,}000$, the reported requirement is 80,600 MB/GPU (629.7 GB aggregate), exceeding available device memory. Full-Shard and Shard+Calib retain distance shards. StreamD+Calib generates distance tiles as required. The work proxy is not a count of distinct moves.